

LAPORAN ASN2 IF12D

PEMROGRAMAN BERORIENTASI OBJEK

Disusun untuk Memenuhi Tugas Mata Kuliah Pemrograman Berorientasi Objek

Dosen Pengampu:

Mutaqin Akbar, S.Kom., M.T., MCE., MCF.



Disusun oleh :

Nama : DIMAS NURDIANSYAH
NIM : 251110058

UNIVERSITAS MERCU BUANA YOGYAKARTA
FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	iii
A. Tugas	1
B. Jawaban.....	1
1. Penjelasan Materi	1
2. Rancangan Program	2
3. Source Code	4
3.1 Class Ruangan.java	4
3.2 Class Kantor.java.....	5
3.3 Class RuangRawatInap.java.....	6
3.4 Class RuangTindakan.java	7
3.5 Class TenagaMedis.java.....	8
3.6 Class Dokter.java	9
3.7 Class Perawat.java	10
3.8 Class Bidan.java	11
3.9 Class TenagaAdministrasi.java.....	12
3.10 Class Departemen.java.....	13
3.11 Class RumahSakit.java.....	14
3.12 Class Main.java	16
4. Output.....	19
5. Penjelasan Kode Program	21
5.1 Class Ruangan.java	21
5.2 Class Kantor.java.....	22
5.3 Class RuangRawatInap.java.....	23
5.4 Class RuangTindakan.java	23
5.5 Class TenagaMedis.java	24
5.6 Class Dokter.java	25
5.7 Class Perawat.java	27
5.8 Class Bidan.java	27

5.9	Class TenagaAdministrasi.java	28
5.10	Class Departemen.java	29
5.11	Class RumahSakit.java	30
5.12	Class Main.java	31
5.13	Konsep PBO	32
6.	Penjelasan Cara Running Program	35
7.	Lampiran	36

DAFTAR GAMBAR

Gambar 3. 1 SourceCode Class Ruangan.java.....	4
Gambar 3. 2 SourceCode Class Kantor.java.....	5
Gambar 3. 3 SourceCode Class RuangRawatInap.java	6
Gambar 3. 4 SourceCode Class RuangTindakan.java	7
Gambar 3. 5 SourceCode Tenaga Medis.java	8
Gambar 3. 6 SourceCode Class Dokter.java.....	9
Gambar 3. 7 SourceCode Class Dokter.java.....	9
Gambar 3. 8 SourceCode Class Perawat.java.....	10
Gambar 3. 9 SourceCode Class Bidan.java	11
Gambar 3. 10 SourceCode Class TenagaAdministrasi.java.....	12
Gambar 3. 11 SourceCode Class Departemen.java	13
Gambar 3. 12 SourceCode Class RumahSakit.java	14
Gambar 3. 13 SourceCode Class RumahSakit.java	15
Gambar 3. 14 SourceCode Class Main.java	16
Gambar 3. 15 SourceCode Class Main.java	17
Gambar 3. 16 SourceCode Class Main.java	18
Gambar 4. 1 Output Program Rumah sakit 1	19
Gambar 4. 2 Output Program Rumah sakit 1	19
Gambar 4. 3 Output Program Rumah sakit 2.....	20
Gambar 4. 4 Output Program Demonstrasi.....	21

A. Tugas

Tema : Rumah Sakit

Membuat program administrasi di rumah sakit dengan beberapa ketentuan berikut:

- a. Terdapat kelas Rumah Sakit, Tenaga Medis, Departemen, Dokter, Perawat, Bidan, Tenaga Administrasi, Ruangan, Kantor, Ruang Rawat Inap, dan Ruang Tindakan.
- b. Dokter, Perawat, Bidan, dan Tenaga Administrasi merupakan subkelas dari Tenaga Medis.
- c. Kantor, Ruang Rawat Inap, dan Ruang Tindakan merupakan subkelas dari Ruangan.
- d. Tentukan atribut dan method yang dimiliki oleh setiap kelas termasuk superkelas dan subkelas.
- e. Rumah Sakit memiliki beberapa Departemen, Ruangan, Dokter, Perawat, Bidan, dan Tenaga Administrasi.
- f. Buatlah profil dari 2 Rumah Sakit (tidak perlu lengkap yang penting tidak terlalu sedikit informasinya).
- g. Semakin banyak konsep PBO yang dipakai, semakin baik nilainya.

B. Jawaban

1. Penjelasan Materi

Pemrograman Berorientasi Objek (PBO) atau *Object Oriented Programming* (OOP) adalah paradigma pemrograman yang mengorganisasi program sebagai kumpulan objek yang saling berinteraksi. Paradigma ini muncul sebagai solusi atas keterbatasan pemrograman prosedural yang semakin sulit dikelola ketika program berkembang menjadi besar dan kompleks. Dalam pemrograman prosedural, program disusun sebagai serangkaian instruksi yang dieksekusi secara berurutan. Pendekatan ini memang sederhana untuk program kecil, namun ketika program membesar, kode menjadi sulit dipahami, sulit diperbaiki, dan sulit dikembangkan. PBO hadir untuk mengatasi masalah ini dengan cara meniru cara pandang dunia nyata — yaitu memandang segala sesuatu sebagai objek.

Dalam PBO, kelas (*class*) berperan sebagai cetakan atau blueprint, sementara objek (*object*) adalah hasil nyata yang dibuat dari cetakan tersebut. Sebagai analogi, jika kelas adalah desain arsitektur sebuah rumah, maka objek adalah rumah-rumah nyata yang dibangun berdasarkan desain tersebut. Satu desain bisa menghasilkan banyak rumah, dan masing-masing rumah bisa memiliki warna cat atau furnitur yang berbeda meskipun strukturnya sama. Setiap objek dalam PBO memiliki dua komponen utama, yaitu atribut yang merupakan data atau informasi yang dimiliki objek tersebut, dan method yang merupakan perilaku atau aksi yang dapat dilakukan oleh objek tersebut. Misalnya, objek Dokter memiliki atribut seperti nama, nip, dan spesialisasi, serta method seperti `periksaPasienn()` dan `buatResep()`.

Selain itu, PBO mendukung penggunaan kembali kode (*code reuse*). Kelas yang sudah dibuat bisa digunakan kembali di program lain tanpa harus menulis ulang dari awal. Dengan inheritance, kelas baru bisa dibangun di atas kelas yang sudah ada, sehingga programmer tidak perlu memulai dari nol setiap kali membutuhkan fungsionalitas yang mirip.

PBO memiliki 4 pilar utama yang menjadi fondasi dalam perancangan program berbasis objek, yaitu:

- Encapsulation => menyembunyikan data internal dan melindunginya dari akses luar yang tidak terkontrol.
- Inheritance => memungkinkan kelas baru mewarisi atribut dan method dari kelas yang sudah ada.
- Polymorphism => memungkinkan satu method atau objek berperilaku berbeda tergantung konteksnya.
- Abstraction => menyembunyikan detail implementasi dan hanya menampilkan fungsionalitas yang diperlukan.

2. Rancangan Program

Berikut adalah tabel penjelasan atribut dan method tiap kelas:

Kelas	Jenis	Atribut	Method
TenagaMedis	abstract	nama, nip, umur, jenisKelamin, jabatan (protected), totalTenagaMedis (static)	getInfoSpesifik() abstract, hitungGaji() abstract, perkenalan(), toString(), getter/setter
Dokter	extends TenagaMedis	spesialisasi, nomorSTR, pengalamanTahun, totalDokter (static)	periksaPasien() overload, perkenalan() override, hitungGaji() override, toString() override
Perawat	extends TenagaMedis	nomorSIP, unitKerja, totalPerawat (static)	rawatPasien(), perkenalan() override, hitungGaji() override, toString() override
Bidan	extends TenagaMedis	nomorSIB, jumlahPeralinan, totalBidan (static)	tanganiPeralinan(), perkenalan() override, hitungGaji() override, toString() override
TenagaAdministrasi	extends TenagaMedis	bagian, keahlian, totalAdmin (static)	inputData(), perkenalan() override, hitungGaji() override, toString() override

Kelas	Jenis	Atribut	Method
Ruangan	abstract	kodeRuangan, namaRuangan, lantai, kapasitas, statusRuangan (protected), totalRuangan (static)	getJenisRuangan() abstract, gunakanRuangan() abstract, toString(), getter/setter
Kantor	extends Ruangan	kepalaKantor, fungsiKantor	gunakanRuangan() override, getJenisRuangan() override, toString() override
RuangRawatInap	extends Ruangan	kelasPerawatan, jumlahTempat, tempatTerisi	gunakanRuangan() override, getJenisRuangan() override, toString() override
RuangTindakan	extends Ruangan	jenisTindakan, peralatanUtama, sedangDigunakan	gunakanRuangan() override, selesaiTindakan(), getJenisRuangan() override, toString() override
Departemen	class biasa	kodeDept, namaDept, kepalaDept, anggota (ArrayList)	tambahAnggota(), tampilkanAnggota(), toString(), getter
RumahSakit	class biasa	namaRS (final), alamat, noTelepon, tipeRS, daftarDokter, daftarPerawat, daftarBidan, daftarAdmin, daftarDepartemen, daftarRuangan (ArrayList)	tambahTenagaMedis() overload, tambahDepartemen(), tambahRuangan(), tampilkanProfil(), toString(), getter

3. Source Code

3.1 Class Ruangan.java

```
Model > Ruangan > Ruangan.java
1  package model.ruangan;
2
3  public abstract class Ruangan {
4
5      private String kodeRuangan;
6      private String namaRuangan;
7      private int    lantai;
8      private int    kapasitas;
9      protected String statusRuangan;
10     private static int totalRuangan = 0;
11
12     public Ruangan(String kodeRuangan, String namaRuangan,
13                    int lantai, int kapasitas) {
14         this.kodeRuangan = kodeRuangan;
15         this.namaRuangan = namaRuangan;
16         this.lantai      = lantai;
17         this.kapasitas   = kapasitas;
18         this.statusRuangan = "Tersedia";
19         totalRuangan++;
20     }
21
22     public abstract String getJenisRuangan();
23     public abstract void  gunakanRuangan();
24
25     public String getKodeRuangan()    { return kodeRuangan; }
26     public String getNamaRuangan()    { return namaRuangan; }
27     public int   getLantai()          { return lantai; }
28     public int   getKapasitas()       { return kapasitas; }
29     public String getStatusRuangan()  { return statusRuangan; }
30     public void  setStatusRuangan(String s) { this.statusRuangan = s; }
31     public static int getTotalRuangan() { return totalRuangan; }
32
33     @Override
34     public String toString() {
35         return "Kode: " + kodeRuangan
36             + " | Nama: " + namaRuangan
37             + " | Lantai: " + lantai
38             + " | Kapasitas: " + kapasitas
39             + " | Status: " + statusRuangan;
40     }
41 }
```

Gambar 3. 1 SourceCode Class Ruangan.java

3.2 Class Kantor.java

```
Model > Ruangan > Kantor.java
1  package model.ruangan;
2
3  public class Kantor extends Ruangan {
4
5      private String kepalaKantor;
6      private String fungsiKantor;
7
8      public Kantor(String kodeRuangan, String namaRuangan, int lantai,
9                     int kapasitas, String kepalaKantor, String fungsiKantor) {
10         super(kodeRuangan, namaRuangan, lantai, kapasitas);
11         this.kepalaKantor = kepalaKantor;
12         this.fungsiKantor = fungsiKantor;
13     }
14
15     @Override
16     public String getJenisRuangan() { return "Kantor"; }
17
18     @Override
19     public void gunakanRuangan() {
20         statusRuangan = "Digunakan";
21         System.out.println(" Kantor [" + getNamaRuangan()
22                             + "] digunakan untuk: " + fungsiKantor);
23     }
24
25     public String getKepalaKantor() { return kepalaKantor; }
26     public String getFungsiKantor() { return fungsiKantor; }
27
28     @Override
29     public String toString() {
30         return super.toString()
31             + "\n        Jenis: " + getJenisRuangan()
32             + " | Kepala: " + kepalaKantor
33             + " | Fungsi: " + fungsiKantor;
34     }
35 }
```

Gambar 3. 2 SourceCode Class Kantor.java

3.3 Class RuangRawatInap.java

```
Model > Ruangan > RuangRawatInap.java
1  package model.ruangan;
2
3  public class RuangRawatInap extends Ruangan {
4
5      private String kelasPerawatan;
6      private int    jumlahTempat;
7      private int    tempatTerisi;
8
9      public RuangRawatInap(String kodeRuangan, String namaRuangan, int lantai,
10                             int kapasitas, String kelasPerawatan, int jumlahTempat) {
11          super(kodeRuangan, namaRuangan, lantai, kapasitas);
12          this.kelasPerawatan = kelasPerawatan;
13          this.jumlahTempat    = jumlahTempat;
14          this.tempatTerisi    = 0;
15      }
16
17      @Override
18      public String getJenisRuangan() { return "Ruang Rawat Inap"; }
19
20      @Override
21      public void gunakanRuangan() {
22          if (tempatTerisi < jumlahTempat) {
23              tempatTerisi++;
24              System.out.println("  Pasien masuk [" + getNamaRuangan() + "]. "
25                                 + "Terisi: " + tempatTerisi + "/" + jumlahTempat);
26          } else {
27              System.out.println("  [" + getNamaRuangan()
28                                 + "] PENUH!");
29          }
30      }
31
32      public String getKelasPerawatan() { return kelasPerawatan; }
33      public int    getJumlahTempat()    { return jumlahTempat; }
34      public int    getTempatTerisi()    { return tempatTerisi; }
35
36      @Override
37      public String toString() {
38          return super.toString()
39                 + "\n        Jenis: " + getJenisRuangan()
40                 + " | Kelas: " + kelasPerawatan
41                 + " | Terisi: " + tempatTerisi + "/" + jumlahTempat;
42      }
43  }
```

Gambar 3. 3 SourceCode Class RuangRawatInap.java

3.4 Class RuangTindakan.java

```
Model > Ruangan > RuangTindakan.java
1  package model.ruangan;
2
3  public class RuangTindakan extends Ruangan {
4
5      private String jenisTindakan;
6      private String peralatanUtama;
7      private boolean sedangDigunakan;
8
9      public RuangTindakan(String kodeRuangan, String namaRuangan, int lantai,
10                          int kapasitas, String jenisTindakan, String peralatanUtama) {
11          super(kodeRuangan, namaRuangan, lantai, kapasitas);
12          this.jenisTindakan = jenisTindakan;
13          this.peralatanUtama = peralatanUtama;
14          this.sedangDigunakan = false;
15      }
16
17      @Override
18      public String getJenisRuangan() { return "Ruang Tindakan"; }
19
20      @Override
21      public void gunakanRuangan() {
22          if (!sedangDigunakan) {
23              sedangDigunakan = true;
24              statusRuangan = "Digunakan";
25              System.out.println(" [" + getNamaRuangan()
26                              + "] digunakan untuk: " + jenisTindakan);
27          } else {
28              System.out.println(" [" + getNamaRuangan() + "] sedang digunakan!");
29          }
30      }
31
32      public void selesaiTindakan() {
33          sedangDigunakan = false;
34          statusRuangan = "Tersedia";
35          System.out.println(" Tindakan di [" + getNamaRuangan()
36                          + "] selesai. Ruangan kembali Tersedia.");
37      }
38
39      public String getJenisTindakan() { return jenisTindakan; }
40      public String getPeralatanUtama() { return peralatanUtama; }
41      public boolean isSedangDigunakan() { return sedangDigunakan; }
42
43      @Override
44      public String toString() {
45          return super.toString()
46              + "\n        Jenis: " + getJenisRuangan()
47              + " | Tindakan: " + jenisTindakan
48              + " | Peralatan: " + peralatanUtama;
49      }
50 }
```

Gambar 3. 4 SourceCode Class RuangTindakan.java

3.5 Class TenagaMedis.java

```
Model > Tenaga > TenagaMedis.java
1 package model.tenaga;
2
3 public abstract class TenagaMedis {
4
5     private String nama;
6     private String nip;
7     private int umur;
8     private String jenisKelamin;
9     protected String jabatan;
10    private static int totalTenagaMedis = 0;
11
12    public TenagaMedis(String nama, String nip, int umur,
13                        String jenisKelamin, String jabatan) {
14        this.nama = nama;
15        this.nip = nip;
16        this.umur = umur;
17        this.jenisKelamin = jenisKelamin;
18        this.jabatan = jabatan;
19        totalTenagaMedis++;
20    }
21
22    Tab to Jump
23    public abstract String getInfoSpesifik();
24    public abstract double hitungGaji();
25
26    public void perkenalan() {
27        System.out.println(" Halo, saya " + nama + ", jabatan: " + jabatan);
28    }
29
30    public String getName() { return nama; }
31    public void setName(String nama) { this.nama = nama; }
32    public String getNip() { return nip; }
33    public void setNip(String nip) { this.nip = nip; }
34    public int getUmur() { return umur; }
35    public void setUmur(int umur) { this.umur = umur; }
36    public String getJenisKelamin() { return jenisKelamin; }
37    public String getJabatan() { return jabatan; }
38    public static int getTotalTenagaMedis() { return totalTenagaMedis; }
39
40    @Override
41    public String toString() {
42        return "Nama: " + nama
43            + " | NIP: " + nip
44            + " | Umur: " + umur
45            + " | " + jenisKelamin
46            + " | Jabatan: " + jabatan;
47    }
48 }
```

Gambar 3. 5 SourceCode Tenaga Medis.java

3.6 Class Dokter.java

```
Model > Tenaga > Dokter.java
1  package model.tenaga;
2
3  public class Dokter extends TenagaMedis {
4
5      private String spesialisasi;
6      private String nomorSTR;
7      private int    pengalamanTahun;
8      private static int totalDokter = 0;
9
10     public Dokter(String nama, String nip, int umur, String jenisKelamin,
11                   String spesialisasi, String nomorSTR, int pengalamanTahun) {
12         super(nama, nip, umur, jenisKelamin, "Dokter");
13         this.spesialisasi = spesialisasi;
14         this.nomorSTR      = nomorSTR;
15         this.pengalamanTahun = pengalamanTahun;
16         totalDokter++;
17     }
18
19     @Override
20     public String getInfoSpesifik() {
21         return "Spesialisasi: " + spesialisasi
22             + " | No. STR: " + nomorSTR
23             + " | Pengalaman: " + pengalamanTahun + " tahun";
24     }
25
26     @Override
27     public double hitungGaji() {
28         double gajiPokok = 8_000_000;
29         double tunjanganSpesialis = spesialisasi.equals("Umum") ? 0 : 3_000_000;
30         double tunjanganPengalaman = pengalamanTahun * 500_000;
31         return gajiPokok + tunjanganSpesialis + tunjanganPengalaman;
32     }
33
34     public void periksaPasien(String namaPasien) {
35         System.out.println(" Dr. " + getName() + " memeriksa: " + namaPasien);
36     }
37
38     public void periksaPasien(String namaPasien, String keluhan) {
39         System.out.println(" Dr. " + getName() + " memeriksa: " + namaPasien
40             + " | Keluhan: " + keluhan);
41     }
42 }
```

Gambar 3. 6 SourceCode Class Dokter.java

```
42
43     @Override
44     public void perkenalan() {
45         System.out.println(" Halo, saya Dr. " + getName()
46             + ", Spesialis " + spesialisasi);
47     }
48
49     public String getSpesialisasi() { return spesialisasi; }
50     public String getNomorSTR()    { return nomorSTR; }
51     public int    getPengalamanTahun() { return pengalamanTahun; }
52     public static int getTotalDokter() { return totalDokter; }
53
54     @Override
55     public String toString() {
56         return super.toString()
57             + "\n          " + getInfoSpesifik()
58             + "\n          Gaji: Rp " + String.format("%.0f", hitungGaji());
59     }
60 }
```

Gambar 3. 7 SourceCode Class Dokter.java

3.7 Class Perawat.java

```
Model > Tenaga > Perawat.java
1 package model.tenaga;
2
3 public class Perawat extends TenagaMedis {
4
5     private String nomorSIP;
6     private String unitKerja;
7     private static int totalPerawat = 0;
8
9     public Perawat(String nama, String nip, int umur, String jenisKelamin,
10                    String nomorSIP, String unitKerja) {
11         super(nama, nip, umur, jenisKelamin, "Perawat");
12         this.nomorSIP = nomorSIP;
13         this.unitKerja = unitKerja;
14         totalPerawat++;
15     }
16
17     @Override
18     public String getInfoSpesifik() {
19         return "No. SIP: " + nomorSIP + " | Unit Kerja: " + unitKerja;
20     }
21
22     @Override
23     public double hitungGaji() {
24         return 5_000_000 + 1_000_000;
25     }
26
27     public void rawatPasien(String namaPasien) {
28         System.out.println(" Perawat " + getName()
29                             + " merawat pasien: " + namaPasien);
30     }
31
32     @Override
33     public void perkenalan() {
34         System.out.println(" Halo, saya Perawat " + getName()
35                             + " di unit " + unitKerja);
36     }
37
38     public String getNomorSIP() { return nomorSIP; }
39     public String getUnitKerja() { return unitKerja; }
40     public static int getTotalPerawat() { return totalPerawat; }
41
42     @Override
43     public String toString() {
44         return super.toString()
45             + "\n          " + getInfoSpesifik()
46             + "\n          Gaji: Rp " + String.format("%.0f", hitungGaji());
47     }
48 }
```

Gambar 3. 8 SourceCode Class Perawat.java

3.8 Class Bidan.java

```
Model > Tenaga > Bidan.java
1 package model.tenaga;
2
3 public class Bidan extends TenagaMedis {
4
5     private String nomorSIB;
6     private int jumlahPersalinan;
7     private static int totalBidan = 0;
8
9     public Bidan(String nama, String nip, int umur,
10                 String jenisKelamin, String nomorSIB) {
11         super(nama, nip, umur, jenisKelamin, "Bidan");
12         this.nomorSIB = nomorSIB;
13         this.jumlahPersalinan = 0;
14         totalBidan++;
15     }
16
17     @Override
18     public String getInfoSpesifik() {
19         return "No. SIB: " + nomorSIB
20             + " | Persalinan Ditangani: " + jumlahPersalinan;
21     }
22
23     @Override
24     public double hitungGaji() {
25         return 5_500_000 + (jumlahPersalinan * 100_000);
26     }
27
28     public void tanganiPersalinan(String namaPasien) {
29         jumlahPersalinan++;
30         System.out.println(" Bidan " + getName()
31             + " menangani persalinan: " + namaPasien);
32     }
33
34     @Override
35     public void perkenalan() {
36         System.out.println(" Halo, saya Bidan " + getName()
37             + ", No. SIB: " + nomorSIB);
38     }
39
40     public String getNomorSIB() { return nomorSIB; }
41     public int getJumlahPersalinan() { return jumlahPersalinan; }
42     public static int getTotalBidan() { return totalBidan; }
43
44     @Override
45     public String toString() {
46         return super.toString()
47             + "\n" + getInfoSpesifik()
48             + "\n" + "Gaji: Rp " + String.format("%.0f", hitungGaji());
49     }
50 }
```

Gambar 3. 9 SourceCode Class Bidan.java

3.9 Class TenagaAdministrasi.java

```
Model > Tenaga > TenagaAdministrasi.java
1  package model.tenaga;
2
3  public class TenagaAdministrasi extends TenagaMedis {
4
5      private String bagian;
6      private String keahlian;
7      private static int totalAdmin = 0;
8
9      public TenagaAdministrasi(String nama, String nip, int umur,
10                               String jenisKelamin,
11                               String bagian, String keahlian) {
12          super(nama, nip, umur, jenisKelamin, "Tenaga Administrasi");
13          this.bagian = bagian;
14          this.keahlian = keahlian;
15          totalAdmin++;
16      }
17
18      @Override
19      public String getInfoSpesifik() {
20          return "Bagian: " + bagian + " | Keahlian: " + keahlian;
21      }
22
23      @Override
24      public double hitungGaji() {
25          return 4_000_000;
26      }
27
28      public void inputData(String jenisData) {
29          System.out.println(" " + getName() + " menginput data: " + jenisData);
30      }
31
32      @Override
33      public void perkenalan() {
34          System.out.println(" Halo, saya " + getName()
35                             + ", bagian " + bagian);
36      }
37
38      public String getBagian() { return bagian; }
39      public String getKeahlian() { return keahlian; }
40      public static int getTotalAdmin() { return totalAdmin; }
41
42      @Override
43      public String toString() {
44          return super.toString()
45                 + "\n      " + getInfoSpesifik()
46                 + "\n      Gaji: Rp " + String.format("%.0f", hitungGaji());
47      }
48  }
```

Gambar 3. 10 SourceCode Class TenagaAdministrasi.java

3.10 Class Departemen.java

```
Model > Departemen.java
1 package model;
2
3 import model.tenaga.TenagaMedis;
4 import java.util.ArrayList;
5
6 public class Departemen {
7
8     private String kodeDept;
9     private String namaDept;
10    private String kepalaDept;
11    private ArrayList<TenagaMedis> anggota;
12
13    public Departemen(String kodeDept, String namaDept, String kepalaDept) {
14        this.kodeDept = kodeDept;
15        this.namaDept = namaDept;
16        this.kepalaDept = kepalaDept;
17        this.anggota = new ArrayList<>();
18    }
19
20    public void tambahAnggota(TenagaMedis tenaga) {
21        anggota.add(tenaga);
22    }
23
24    public void tampilkanAnggota() {
25        System.out.println("    Dept [" + namaDept
26            + "] | Kepala: " + kepalaDept
27            + " | Anggota: " + anggota.size());
28        for (TenagaMedis t : anggota) {
29            System.out.println("        - " + t.getNama()
30                + " (" + t.getJabatan() + ")");
31        }
32    }
33
34    public String getKodeDept() { return kodeDept; }
35    public String getNamaDept() { return namaDept; }
36    public String getKepalaDept() { return kepalaDept; }
37    public ArrayList<TenagaMedis> getAnggota() { return anggota; }
38
39    @Override
40    public String toString() {
41        return "Kode: " + kodeDept + " | Nama: " + namaDept
42            + " | Kepala: " + kepalaDept + " | Anggota: " + anggota.size();
43    }
44 }
```

Gambar 3. 11 SourceCode Class Departemen.java

3.11 Class RumahSakit.java

```
Model > RumahSakit.java
1 package model;
2
3 import model.tenaga.*;
4 import model.ruangan.*;
5 import java.util.ArrayList;
6
7 public class RumahSakit {
8
9     private final String namaRS;
10    private String alamat;
11    private String noTelepon;
12    private String tipeRS;
13
14    private ArrayList<Departemen> daftarDepartemen;
15    private ArrayList<Ruangan> daftarRuangan;
16    private ArrayList<Dokter> daftarDokter;
17    private ArrayList<Perawat> daftarPerawat;
18    private ArrayList<Bidan> daftarBidan;
19    private ArrayList<TenagaAdministrasi> daftarAdmin;
20
21    public RumahSakit(String namaRS, String alamat,
22                      String noTelepon, String tipeRS) {
23        this.namaRS = namaRS;
24        this.alamat = alamat;
25        this.noTelepon = noTelepon;
26        this.tipeRS = tipeRS;
27        daftarDepartemen = new ArrayList<>();
28        daftarRuangan = new ArrayList<>();
29        daftarDokter = new ArrayList<>();
30        daftarPerawat = new ArrayList<>();
31        daftarBidan = new ArrayList<>();
32        daftarAdmin = new ArrayList<>();
33    }
34
35    public void tambahTenagaMedis(Dokter d) { daftarDokter.add(d); }
36    public void tambahTenagaMedis(Perawat p) { daftarPerawat.add(p); }
37    public void tambahTenagaMedis(Bidan b) { daftarBidan.add(b); }
38    public void tambahTenagaMedis(TenagaAdministrasi ta) { daftarAdmin.add(ta); }
39
40    public void tambahDepartemen(Departemen d) { daftarDepartemen.add(d); }
41    public void tambahRuangan(Ruangan r) { daftarRuangan.add(r); }
42}
```

Gambar 3. 12 SourceCode Class RumahSakit.java

```

Model > RumahSakit.java
7 public class RumahSakit {
42
43     public void tampilkanProfil() {
44         System.out.println("\n" + "=".repeat(65));
45         System.out.println("    PROFIL RUMAH SAKIT: " + namaRS);
46         System.out.println("=".repeat(65));
47         System.out.println("    Alamat      : " + alamat);
48         System.out.println("    No. Telepon : " + noTelepon);
49         System.out.println("    Tipe RS     : " + tipeRS);
50         System.out.println("    Departemen  : " + daftarDepartemen.size());
51         System.out.println("    Total Ruangan : " + daftarRuangan.size());
52         System.out.println("    Dokter      : " + daftarDokter.size());
53         System.out.println("    Perawat     : " + daftarPerawat.size());
54         System.out.println("    Bidan       : " + daftarBidan.size());
55         System.out.println("    Tenaga Admin : " + daftarAdmin.size());
56
57         System.out.println("\n -- DAFTAR DEPARTEMEN --");
58         for (Departemen d : daftarDepartemen) d.tampilkanAnggota();
59
60         System.out.println("\n -- DAFTAR DOKTER --");
61         for (Dokter d : daftarDokter)
62             System.out.println(" > " + d + "\n");
63
64         System.out.println(" -- DAFTAR PERAWAT --");
65         for (Perawat p : daftarPerawat)
66             System.out.println(" > " + p + "\n");
67
68         System.out.println(" -- DAFTAR BIDAN --");
69         for (Bidan b : daftarBidan)
70             System.out.println(" > " + b + "\n");
71
72         System.out.println(" -- DAFTAR TENAGA ADMINISTRASI --");
73         for (TenagaAdministrasi ta : daftarAdmin)
74             System.out.println(" > " + ta + "\n");
75
76         System.out.println(" -- DAFTAR RUANGAN --");
77         for (Ruangan r : daftarRuangan)
78             System.out.println(" > " + r + "\n");
79     }
80
81     public String getNamaRS() { return namaRS; }
82     public String getAlamat() { return alamat; }
83     public String getNoTelepon() { return noTelepon; }
84     public String getTipeRS() { return tipeRS; }
85
86     @Override
87     public String toString() {
88         return "RS: " + namaRS + " | Alamat: " + alamat + " | Tipe: " + tipeRS;
89     }
90 }

```

Gambar 3. 13 SourceCode Class RumahSakit.java

3.12 Class Main.java

```
Main.java
1  import model.*;
2  import model.tenaga.*;
3  import model.ruangan.*;
4
5  public class Main {
6      public static void main(String[] args) {
7          // Ctrl+L for Agent
8          // RUMAH SAKIT 1 - RS Umum Pusat Dr. Sardjito
9
10         RumahSakit rs1 = new RumahSakit(
11             "RS Umum Pusat Dr. Sardjito",
12             "Jl. Kesehatan No.1, Yogyakarta",
13             "(0274) 587333", "Tipe A"
14         );
15
16         Dokter dr1 = new Dokter("Budi Santoso", "NIP001", 45, "Laki-laki", "Kardiologi", "STR-001", 15);
17         Dokter dr2 = new Dokter("Siti Aminah", "NIP002", 38, "Perempuan", "Pediatri", "STR-002", 8);
18         Dokter dr3 = new Dokter("Ahmad Fauzi", "NIP003", 50, "Laki-laki", "Umum", "STR-003", 20);
19
20         Perawat pr1 = new Perawat("Dewi Lestari", "NIP004", 30, "Perempuan", "SIP-001", "ICU");
21         Perawat pr2 = new Perawat("Rudi Hartono", "NIP005", 27, "Laki-laki", "SIP-002", "UGD");
22
23         Bidan bd1 = new Bidan("Rina Kusuma", "NIP006", 35, "Perempuan", "SIB-001");
24         bd1.tanganiPersalinan("Ny. Sari");
25         bd1.tanganiPersalinan("Ny. Dewi");
26
27         TenagaAdministrasi ta1 = new TenagaAdministrasi(
28             "Andi Wijaya", "NIP007", 28, "Laki-laki", "Keuangan", "Akuntansi");
29
30         Departemen dept1 = new Departemen("D001", "Kardiologi", "Dr. Budi Santoso");
31         dept1.tambahAnggota(dr1); dept1.tambahAnggota(pr1);
32
33         Departemen dept2 = new Departemen("D002", "Pediatri", "Dr. Siti Aminah");
34         dept2.tambahAnggota(dr2); dept2.tambahAnggota(pr2);
35
36         Kantor kt1 = new Kantor("KT01", "Kantor Direktur", 1, 10, "Dr. Budi", "Manajemen RS");
37         RuangRawatInap rri1 = new RuangRawatInap("RI01", "Ruang Mawar", 2, 4, "VIP", 4);
38         RuangRawatInap rri2 = new RuangRawatInap("RI02", "Ruang Melati", 2, 6, "Kelas 1", 6);
39         RuangTindakan rtl = new RuangTindakan("RT01", "Ruang Operasi 1", 3, 5, "Operasi Mayor", "Alat Bedah");
40
41         rs1.tambahTenagaMedis(dr1); rs1.tambahTenagaMedis(dr2); rs1.tambahTenagaMedis(dr3);
42         rs1.tambahTenagaMedis(pr1); rs1.tambahTenagaMedis(pr2);
43         rs1.tambahTenagaMedis(bd1); rs1.tambahTenagaMedis(ta1);
44         rs1.tambahDepartemen(dept1); rs1.tambahDepartemen(dept2);
45         rs1.tambahRuangan(kt1); rs1.tambahRuangan(rri1);
46         rs1.tambahRuangan(rri2); rs1.tambahRuangan(rtl);
47     }
```

Gambar 3. 14 SourceCode Class Main.java

```

Main.java
5  public class Main {
6      public static void main(String[] args) {
47
48          // RUMAH SAKIT 2 - RS PKU Muhammadiyah
49
50          RumahSakit rs2 = new RumahSakit(
51              "RS PKU Muhammadiyah",
52              "Jl. Ahmad Dahlan No.20, Yogyakarta",
53              "(0274) 512653", "Tipe B"
54          );
55
56          Dokter dr4 = new Dokter("Lisa Permata","NIP008",42,"Perempuan","Neurologi","STR-004",12);
57          Dokter dr5 = new Dokter("Hendra Putra","NIP009",36,"Laki-laki","Ortopedi", "STR-005", 6);
58
59          Perawat pr3 = new Perawat("Maya Sari","NIP010",25,"Perempuan","SIP-003","Neurologi");
60
61          Bidan bd2 = new Bidan("Nur Hasanah","NIP011",33,"Perempuan","SIB-002");
62          bd2.tanganiPersalinan("Ny. Fitri");
63
64          TenagaAdministrasi ta2 = new TenagaAdministrasi(
65              "Bayu Setiawan","NIP012",31,"Laki-laki","Rekam Medis","Sistem Informasi");
66
67          Departemen dept3 = new Departemen("D003","Neurologi","Dr. Lisa Permata");
68          dept3.tambahAnggota(dr4); dept3.tambahAnggota(pr3);
69
70          Kantor      kt2  = new Kantor("KT02","Kantor Administrasi",1,8,"Bayu Setiawan","Administrasi");
71          RuangRawatInap rri3 = new RuangRawatInap("RI03","Ruang Kenanga",2,4,"Kelas 2",4);
72          RuangTindakan rt2 = new RuangTindakan("RT02","Ruang ICU",3,6,"Perawatan Intensif","Ventilator");
73
74          rs2.tambahTenagaMedis(dr4); rs2.tambahTenagaMedis(dr5);
75          rs2.tambahTenagaMedis(pr3); rs2.tambahTenagaMedis(bd2);
76          rs2.tambahTenagaMedis(ta2); rs2.tambahDepartemen(dept3);
77          rs2.tambahRuangan(kt2); rs2.tambahRuangan(rri3); rs2.tambahRuangan(rt2);
78
79          // TAMPILKAN PROFIL KEDUA RS
80
81          rs1.tampilkanProfil();
82          rs2.tampilkanProfil();

```

Gambar 3. 15 SourceCode Class Main.java

```

Main.java
5  public class Main {
6      public static void main(String[] args) {
86
87          System.out.println("\n" + "=".repeat(65));
88          System.out.println(" DEMONSTRASI KONSEP PBO");
89          System.out.println("=".repeat(65));
90
91          System.out.println("\n[1] POLYMORPHISM - Override perkenalan()");
92          TenagaMedis[] semuaTenaga = {dr1, pr1, bd1, tal};
93          for (TenagaMedis t : semuaTenaga) t.perkenalan();
94
95          System.out.println("\n[2] METHOD OVERLOADING - periksaPasien()");
96          dr1.periksaPasien("Pak Joko");
97          dr1.periksaPasien("Pak Joko", "Nyeri Dada");
98
99          System.out.println("\n[3] POLYMORPHISM - gunakanRuangan()");
100         Ruangan[] semuaRuangan = {kt1, rr1l, rtl};
101         for (Ruangan r : semuaRuangan) r.gunakanRuangan();
102
103         System.out.println("\n[4] RuangRawatInap - isi sampai penuh");
104         rr1l.gunakanRuangan(); rr1l.gunakanRuangan();
105         rr1l.gunakanRuangan(); rr1l.gunakanRuangan();
106
107         System.out.println("\n[5] RuangTindakan - selesaiTindakan()");
108         rtl.gunakanRuangan();
109         rtl.selesaiTindakan();
110         rtl.gunakanRuangan();
111
112         System.out.println("\n[6] STATIC - Total objek yang dibuat:");
113         System.out.println(" Total TenagaMedis : " + TenagaMedis.getTotalTenagaMedis());
114         System.out.println(" Total Dokter      : " + Dokter.getTotalDokter());
115         System.out.println(" Total Perawat   : " + Perawat.getTotalPerawat());
116         System.out.println(" Total Bidan     : " + Bidan.getTotalBidan());
117         System.out.println(" Total Admin     : " + TenagaAdministrasi.getTotalAdmin());
118         System.out.println(" Total Ruangan   : " + Ruangan.getTotalRuangan());
119
120         System.out.println("\n[7] FINAL - Nama RS tidak bisa diubah:");
121         System.out.println(" RS1: " + rs1.getNamaRS());
122         System.out.println(" RS2: " + rs2.getNamaRS());
123
124         System.out.println("\n" + "=".repeat(65));
125         System.out.println(" SELESAI!");
126         System.out.println("=".repeat(65));
127     }
128 }

```

Gambar 3. 16 SourceCode Class Main.java

4. Output

```
C:\Users\Bestdym\Data\Praktikum\TugasPBO>javac Model/Ruangan/*.java Model/Tenaga/*.java Model/*.java Main.java
C:\Users\Bestdym\Data\Praktikum\TugasPBO>java Main
Bidan Rina Kusuma menangani persalinan: Ny. Sari
Bidan Rina Kusuma menangani persalinan: Ny. Dewi
Bidan Nur Hasanah menangani persalinan: Ny. Fitri

=====
PROFIL RUMAH SAKIT: RS Umum Pusat Dr. Sardjito
=====
Alamat      : Jl. Kesehatan No.1, Yogyakarta
No. Telepon  : (0274) 587333
Tipe RS     : Tipe A
Departemen  : 2
Total Ruangan : 4
Dokter     : 3
Perawat    : 2
Bidan      : 1
Tenaga Admin : 1

-- DAFTAR DEPARTEMEN --
Dept [Kardiologi] | Kepala: Dr. Budi Santoso | Anggota: 2
- Budi Santoso (Dokter)
- Dewi Lestari (Perawat)
Dept [Pediatri] | Kepala: Dr. Siti Aminah | Anggota: 2
- Siti Aminah (Dokter)
- Rudi Hartono (Perawat)

-- DAFTAR DOKTER --
> Nama: Budi Santoso | NIP: NIP001 | Umur: 45 | Laki-laki | Jabatan: Dokter
  Spesialisasi: Kardiologi | No. STR: STR-001 | Pengalaman: 15 tahun
  Gaji: Rp 18.500.000

> Nama: Siti Aminah | NIP: NIP002 | Umur: 38 | Perempuan | Jabatan: Dokter
  Spesialisasi: Pediatri | No. STR: STR-002 | Pengalaman: 8 tahun
  Gaji: Rp 15.000.000

> Nama: Ahmad Fauzi | NIP: NIP003 | Umur: 50 | Laki-laki | Jabatan: Dokter
  Spesialisasi: Umum | No. STR: STR-003 | Pengalaman: 20 tahun
  Gaji: Rp 18.000.000

-- DAFTAR PERAWAT --
> Nama: Dewi Lestari | NIP: NIP004 | Umur: 30 | Perempuan | Jabatan: Perawat
  No. SIP: SIP-001 | Unit Kerja: ICU
  Gaji: Rp 6.000.000

> Nama: Rudi Hartono | NIP: NIP005 | Umur: 27 | Laki-laki | Jabatan: Perawat
  No. SIP: SIP-002 | Unit Kerja: UGD
  Gaji: Rp 6.000.000
```

Gambar 4. 1 Output Program Rumah sakit 1

```
-- DAFTAR BIDAN --
> Nama: Rina Kusuma | NIP: NIP006 | Umur: 35 | Perempuan | Jabatan: Bidan
  No. SIB: SIB-001 | Persalinan Ditangani: 2
  Gaji: Rp 5.700.000

-- DAFTAR TENAGA ADMINISTRASI --
> Nama: Andi Wijaya | NIP: NIP007 | Umur: 28 | Laki-laki | Jabatan: Tenaga Administrasi
  Bagian: Keuangan | Keahlian: Akuntansi
  Gaji: Rp 4.000.000

-- DAFTAR RUANGAN --
> Kode: KT01 | Nama: Kantor Direktur | Lantai: 1 | Kapasitas: 10 | Status: Tersedia
  Jenis: Kantor | Kepala: Dr. Budi | Fungsi: Manajemen RS

> Kode: RI01 | Nama: Ruang Mawar | Lantai: 2 | Kapasitas: 4 | Status: Tersedia
  Jenis: Ruang Rawat Inap | Kelas: VIP | Terisi: 0/4

> Kode: RI02 | Nama: Ruang Melati | Lantai: 2 | Kapasitas: 6 | Status: Tersedia
  Jenis: Ruang Rawat Inap | Kelas: Kelas 1 | Terisi: 0/6

> Kode: RT01 | Nama: Ruang Operasi 1 | Lantai: 3 | Kapasitas: 5 | Status: Tersedia
  Jenis: Ruang Tindakan | Tindakan: Operasi Mayor | Peralatan: Alat Bedah
```

Gambar 4. 2 Output Program Rumah sakit 1

```

=====
PROFIL RUMAH SAKIT: RS PKU Muhammadiyah
=====
Alamat      : Jl. Ahmad Dahlan No.20, Yogyakarta
No. Telepon  : (0274) 512653
Tipe RS     : Tipe B
Departemen  : 1
Total Ruangan : 3
Dokter      : 2
Perawat     : 1
Bidan       : 1
Tenaga Admin : 1

-- DAFTAR DEPARTEMEN --
Dept [Neurologi] | Kepala: Dr. Lisa Permata | Anggota: 2
- Lisa Permata (Dokter)
- Maya Sari (Perawat)

-- DAFTAR DOKTER --
> Nama: Lisa Permata | NIP: NIP008 | Umur: 42 | Perempuan | Jabatan: Dokter
  Spesialisasi: Neurologi | No. STR: STR-004 | Pengalaman: 12 tahun
  Gaji: Rp 17.000.000

> Nama: Hendra Putra | NIP: NIP009 | Umur: 36 | Laki-laki | Jabatan: Dokter
  Spesialisasi: Ortopedi | No. STR: STR-005 | Pengalaman: 6 tahun
  Gaji: Rp 14.000.000

-- DAFTAR PERAWAT --
> Nama: Maya Sari | NIP: NIP010 | Umur: 25 | Perempuan | Jabatan: Perawat
  No. SIP: SIP-003 | Unit Kerja: Neurologi
  Gaji: Rp 6.000.000

-- DAFTAR BIDAN --
> Nama: Nur Hasanah | NIP: NIP011 | Umur: 33 | Perempuan | Jabatan: Bidan
  No. SIB: SIB-002 | Persalinan Ditangani: 1
  Gaji: Rp 5.600.000

-- DAFTAR TENAGA ADMINISTRASI --
> Nama: Bayu Setiawan | NIP: NIP012 | Umur: 31 | Laki-laki | Jabatan: Tenaga Administrasi
  Bagian: Rekam Medis | Keahlian: Sistem Informasi
  Gaji: Rp 4.000.000

-- DAFTAR RUANGAN --
> Kode: KT02 | Nama: Kantor Administrasi | Lantai: 1 | Kapasitas: 8 | Status: Tersedia
  Jenis: Kantor | Kepala: Bayu Setiawan | Fungsi: Administrasi

> Kode: RI03 | Nama: Ruang Kenanga | Lantai: 2 | Kapasitas: 4 | Status: Tersedia
  Jenis: Ruang Rawat Inap | Kelas: Kelas 2 | Terisi: 0/4

> Kode: RT02 | Nama: Ruang ICU | Lantai: 3 | Kapasitas: 6 | Status: Tersedia
  Jenis: Ruang Tindakan | Tindakan: Perawatan Intensif | Peralatan: Ventilator

```

Gambar 4. 3 Output Program Rumah sakit 2


```

=====
DEMONSTRASI KONSEP PBO
=====

[1] POLYMORPHISM - Override perkenalan()
Halo, saya Dr. Budi Santoso, Spesialis Kardiologi
Halo, saya Perawat Dewi Lestari di unit ICU
Halo, saya Bidan Rina Kusuma, No. SIB: SIB-001
Halo, saya Andi Wijaya, bagian Keuangan

[2] METHOD OVERLOADING - periksaPasien()
Dr. Budi Santoso memeriksa: Pak Joko
Dr. Budi Santoso memeriksa: Pak Joko | Keluhan: Nyeri Dada

[3] POLYMORPHISM - gunakanRuangan()
Kantor [Kantor Direktur] digunakan untuk: Manajemen RS
Pasien masuk [Ruang Mawar]. Terisi: 1/4
[Ruang Operasi 1] digunakan untuk: Operasi Mayor

[4] RuangRawatInap - isi sampai penuh
Pasien masuk [Ruang Mawar]. Terisi: 2/4
Pasien masuk [Ruang Mawar]. Terisi: 3/4
Pasien masuk [Ruang Mawar]. Terisi: 4/4
[Ruang Mawar] PENUH!

[5] RuangTindakan - selesaiTindakan()
[Ruang Operasi 1] sedang digunakan!
Tindakan di [Ruang Operasi 1] selesai. Ruangan kembali Tersedia.
[Ruang Operasi 1] digunakan untuk: Operasi Mayor

[6] STATIC - Total objek yang dibuat:
Total TenagaMedis : 12
Total Dokter      : 5
Total Perawat     : 3
Total Bidan       : 2
Total Admin       : 2
Total Ruangan     : 7

[7] FINAL - Nama RS tidak bisa diubah:
RS1: RS Umum Pusat Dr. Sardjito
RS2: RS PKU Muhammadiyah

=====
SELESAI!
=====

```

Gambar 4. 4 Output Program Demonstrasi

5. Penjelasan Kode Program

5.1 Class Ruangan.java

Class Ruangan adalah kelas abstrak yang berfungsi sebagai superclass untuk semua jenis ruangan di rumah sakit. Polanya serupa dengan TenagaMedis namun dikhususkan untuk entitas ruangan.

Penjelasan Kode Program:

- `package model.ruangan` => berada di folder model/ruangan, dikelompokkan terpisah dari package tenaga karena merepresentasikan entitas yang berbeda.
- `public abstract class Ruangan` => kelas abstrak yang tidak bisa di-instantiasi langsung. Hanya bisa digunakan melalui subclassnya (Kantor, RuangRawatInap, RuangTindakan).
- `private String kodeRuangan, private String namaRuangan, private int lantai, private int kapasitas` => atribut dasar semua ruangan bersifat private sebagai penerapan enkapsulasi. int digunakan untuk lantai dan kapasitas karena berupa bilangan bulat.
- `protected String statusRuangan` => dibuat protected agar subclass bisa mengubah nilai status secara langsung tanpa setter, misalnya `statusRuangan = "Digunakan"` di class Kantor dan RuangTindakan.

- `private static int totalRuangan = 0` => counter static untuk menghitung total semua ruangan yang dibuat dari semua jenis subclass.
- `public Ruangan(String kodeRuangan, ...) { this.statusRuangan = "Tersedia"; totalRuangan++; }` => Constructor menginisialisasi semua atribut menggunakan `this`, menetapkan status awal "Tersedia" untuk setiap ruangan baru, dan menambah counter.
- `public abstract String getJenisRuangan()` dan `public abstract void gunakanRuangan()` => dua method abstrak wajib diimplementasikan oleh setiap subclass. `getJenisRuangan()` mengembalikan String nama jenis ruangan, sedangkan `gunakanRuangan()` bertipe void karena hanya menjalankan aksi.
- **Getter dan setter** => `setStatusRuangan(String s)` adalah satu-satunya setter yang tersedia, memungkinkan status ruangan diubah dari luar. Atribut lain hanya memiliki getter sehingga tidak bisa diubah setelah objek dibuat.
- `public static int getTotalRuangan()` => method static untuk mengambil nilai counter total ruangan, dipanggil dengan `Ruangan.getTotalRuangan()`.
- `@Override public String toString()` => menampilkan info dasar ruangan: kode, nama, lantai, kapasitas, dan status dalam satu baris teks.

5.2 Class Kantor.java

Class Kantor adalah subclass dari Ruangan yang merepresentasikan ruangan berjenis kantor di rumah sakit.

Penjelasan Kode Program:

- `package model.ruangan` => berada di package yang sama dengan Ruangan, tidak perlu import.
- `public class Kantor extends Ruangan` => mewarisi Ruangan dan wajib mengimplementasikan `getJenisRuangan()` dan `gunakanRuangan()`.
- `private String kepalaKantor, private String fungsiKantor` => atribut tambahan khusus Kantor. `kepalaKantor` menyimpan nama penanggung jawab kantor, `fungsiKantor` menyimpan fungsi atau peruntukan kantor tersebut.
- `public Kantor(String kodeRuangan, ...) { super(kodeRuangan, namaRuangan, lantai, kapasitas); }` => Constructor memanggil `super()` untuk menginisialisasi atribut Ruangan, lalu mengisi atribut khusus Kantor dengan `this`.
- `@Override public String getJenisRuangan()` => implementasi wajib yang langsung mengembalikan String "Kantor" sebagai identitas jenis ruangan.
- `@Override public void gunakanRuangan()` => implementasi wajib. Di sinilah atribut `protected statusRuangan` dari superclass langsung diubah menjadi "Digunakan" tanpa perlu setter, karena `protected` membolehkan akses langsung dari subclass.
- `public String getKepalaKantor(), public String getFungsiKantor()` => getter untuk atribut private khusus Kantor.

- `@Override public String toString()` => memanggil `super.toString()` untuk mendapatkan info dasar ruangan, lalu menambahkan jenis ruangan, nama kepala kantor, dan fungsinya.

5.3 Class RuangRawatInap.java

Class `RuangRawatInap` adalah subclass `Ruangan` yang merepresentasikan ruang perawatan pasien. Kelas ini memiliki logika khusus untuk memantau kapasitas tempat tidur secara real-time.

Penjelasan Kode Program:

- `package model.ruangan` => berada di package yang sama dengan `Ruangan`.
- `public class RuangRawatInap extends Ruangan` => mewarisi `Ruangan` dan wajib mengimplementasikan kedua method abstraknya.
- `private String kelasPerawatan, private int jumlahTempat, private int tempatTerisi` => atribut khusus rawat inap. `kelasPerawatan` menyimpan kelas seperti "VIP", "Kelas 1", dst. `jumlahTempat` adalah kapasitas maksimal, `tempatTerisi` adalah jumlah tempat yang sudah terisi, diinisialisasi dengan 0.
- `public RuangRawatInap(String kodeRuangan, ...) { super(...); this.tempatTerisi = 0; }` => Constructor memanggil `super()` lalu mengisi atribut khusus. `tempatTerisi` di-set ke 0 karena ruangan baru belum terisi.
- `@Override public String getJenisRuangan()` => mengembalikan String "Ruang Rawat Inap".
- `@Override public void gunakanRuangan()` => implementasi dengan logika kondisional menggunakan if-else. Jika `tempatTerisi < jumlahTempat`, maka `tempatTerisi++` dan pasien berhasil masuk. Jika sudah penuh, mencetak pesan "PENUH!". Ini membuat simulasi ruangan yang realistis.
- Getter untuk `kelasPerawatan, jumlahTempat, tempatTerisi` => tidak ada setter untuk ketiganya karena nilai dikelola secara internal oleh method `gunakanRuangan()`.
- `@Override public String toString()` => memanggil `super.toString()` dan menambahkan jenis ruangan, kelas perawatan, serta informasi kapasitas terisi dalam format terisi/total.

5.4 Class RuangTindakan.java

Class `RuangTindakan` adalah subclass `Ruangan` yang merepresentasikan ruang untuk prosedur medis. Keunikannya adalah menggunakan tipe data boolean untuk memantau apakah ruangan sedang digunakan atau tidak.

Penjelasan Kode Program:

- `package model.ruangan` => berada di package yang sama dengan `Ruangan`.
- `public class RuangTindakan extends Ruangan` => mewarisi `Ruangan` dan wajib mengimplementasikan kedua method abstraknya.

- `private String jenisTindakan, private String peralatanUtama, private boolean sedangDigunakan` => atribut khusus RuangTindakan. `jenisTindakan` menyimpan jenis prosedur seperti "Operasi Mayor". `peralatanUtama` menyimpan alat utama yang tersedia. `boolean sedangDigunakan` hanya bernilai true atau false, diinisialisasi false karena ruangan awalnya belum digunakan.
- `public RuangTindakan(String kodeRuangan, ...) { super(...); this.sedangDigunakan = false; }` => Constructor memanggil `super()` dan mengisi atribut khusus. `sedangDigunakan` di-set false sebagai kondisi awal.
- `@Override public String getJenisRuangan()` => mengembalikan String "Ruang Tindakan".
- `@Override public void gunakanRuangan()` => menggunakan `if (!sedangDigunakan)` — tanda ! berarti "jika TIDAK sedang digunakan". Jika kondisi terpenuhi, `sedangDigunakan` diubah ke true dan `statusRuangan` (atribut protected dari RuangTindakan) diubah ke "Digunakan". Jika sudah digunakan, mencetak pesan bahwa ruangan sedang terpakai.
- `public void selesaiTindakan()` => method unik yang mereset `sedangDigunakan` kembali ke false dan `statusRuangan` kembali ke "Tersedia". Memungkinkan ruangan digunakan kembali setelah tindakan selesai.
- `public boolean isSedangDigunakan()` => getter untuk atribut boolean. Nama getter untuk tipe boolean biasanya menggunakan prefix `is` bukan `get` sesuai konvensi Java.
- `@Override public String toString()` => memanggil `super.toString()` dan menambahkan jenis tindakan serta peralatan utama.

5.5 Class TenagaMedis.java

Class `TenagaMedis` adalah kelas abstrak yang berfungsi sebagai superclass utama dalam hierarki tenaga medis. Kelas ini tidak bisa dibuat objeknya secara langsung karena bersifat `abstract`, dan wajib diwarisi oleh subclass-nya.

Penjelasan Kode Program:

- `package model.tenaga` => menandakan bahwa file ini berada di dalam folder `model/tenaga`. Package berfungsi mengelompokkan kelas-kelas yang saling berkaitan agar kode lebih terorganisir dan menghindari konflik nama antar kelas.
- `public abstract class TenagaMedis` => mendeklarasikan kelas ini bersifat abstrak. Keyword `public` berarti kelas ini bisa diakses dari package manapun, sedangkan `abstract` berarti kelas ini hanya berfungsi sebagai kerangka dan tidak bisa diinstansiasi langsung menggunakan `new TenagaMedis()`.
- `private String nama, private String nip, private int umur, private String jenisKelamin` => atribut-atribut dengan access modifier `private`, artinya hanya bisa diakses dari dalam class `TenagaMedis` sendiri. Tipe data `String` digunakan untuk data teks, sedangkan `int` untuk bilangan bulat (umur).

- `protected String jabatan` => atribut dengan access modifier `protected`, artinya bisa diakses langsung oleh subclass (Dokter, Perawat, Bidan, TenagaAdministrasi) maupun kelas lain dalam package yang sama, tanpa perlu melalui getter.
- `private static int totalTenagaMedis = 0` => atribut static yang dimiliki oleh kelas, bukan oleh objek. Nilai ini dibagi bersama oleh seluruh objek TenagaMedis sehingga berfungsi sebagai penghitung total tenaga medis yang dibuat. Diinisialisasi dengan nilai 0.
- `public TenagaMedis(String nama, String nip, int umur, String jenisKelamin, String jabatan)` => Constructor kelas ini. Dipanggil otomatis saat objek subclass dibuat. Keyword `this.nama = nama` digunakan untuk membedakan antara atribut kelas (`this.nama`) dengan parameter yang masuk (`nama`) karena keduanya memiliki nama yang sama. Setiap kali constructor ini dipanggil, `totalTenagaMedis++` dijalankan sehingga counter bertambah otomatis.
- `public abstract String getInfoSpesifik()` dan `public abstract double hitungGaji()` => dua method abstrak tanpa body/isi yang wajib diimplementasikan oleh seluruh subclass. `String` berarti method harus mengembalikan nilai teks, `double` berarti harus mengembalikan bilangan desimal (untuk nilai gaji).
- `public void perkenalan()` => method konkret yang sudah memiliki isi. Bertipe `void` karena tidak mengembalikan nilai. Method ini bisa di-override oleh subclass untuk menampilkan format perkenalan yang berbeda.
- `public String getNama(), public void setNama(), dst` => method getter dan setter sebagai penerapan konsep Encapsulation. Getter bertipe sesuai atributnya (`String`, `int`) untuk membaca nilai atribut private. Setter bertipe `void` untuk mengubah nilai atribut private dari luar kelas. `getJenisKelamin()` hanya memiliki getter tanpa setter, artinya jenis kelamin tidak bisa diubah setelah objek dibuat.
- `public static int getTotalTenagaMedis()` => method getter untuk atribut static `totalTenagaMedis`. Dibuat static juga karena atribut yang diaksesnya bersifat static, sehingga bisa dipanggil langsung dari nama kelas tanpa membuat objek: `TenagaMedis.getTotalTenagaMedis()`.
- `@Override public String toString()` => meng-override method `toString()` bawaan Java agar ketika objek dicetak dengan `System.out.println()`, tampilannya berupa informasi yang informatif bukan kode memori default Java. Mengembalikan kombinasi semua atribut dalam format teks yang rapi.

5.6 Class Dokter.java

Class Dokter adalah subclass dari TenagaMedis yang merepresentasikan tenaga medis berjenis dokter. Kelas ini mengimplementasikan seluruh method abstract dari TenagaMedis dan menambahkan atribut serta method yang spesifik untuk dokter.

Penjelasan Kode Program:

- `package model.tenaga` => berada dalam package yang sama dengan TenagaMedis sehingga dapat langsung mengakses TenagaMedis tanpa perlu import.

- `public class Dokter extends TenagaMedis` => mendeklarasikan bahwa Dokter mewarisi TenagaMedis menggunakan keyword `extends`. Dokter secara otomatis mendapatkan semua atribut dan method dari TenagaMedis, dan wajib mengimplementasikan kedua method abstraknya.
- `private String spesialisasi, private String nomorSTR, private int pengalamanTahun` => atribut tambahan khusus milik Dokter yang tidak ada di TenagaMedis. Semua `private` sebagai penerapan enkapsulasi. `nomorSTR` adalah Surat Tanda Registrasi sebagai nomor izin praktik dokter. `int pengalamanTahun` untuk menyimpan lama pengalaman dalam satuan tahun.
- `private static int totalDokter = 0` => counter khusus untuk menghitung jumlah objek Dokter saja, terpisah dari `totalTenagaMedis` di superclass.
- `public Dokter(String nama, ...) { super(...); }` => Constructor Dokter memanggil `super(nama, nip, umur, jenisKelamin, "Dokter")` di baris pertama untuk menginisialisasi atribut yang diwarisi dari TenagaMedis. Nilai "Dokter" langsung di-hardcode sebagai jabatan. Setelah `super()`, atribut khusus Dokter diisi menggunakan `this.totalDokter++` dijalankan setiap kali objek Dokter baru dibuat.
- `@Override public String getInfoSpesifik()` => implementasi wajib dari method abstract TenagaMedis. Mengembalikan String berisi informasi spesifik dokter yaitu spesialisasi, nomor STR, dan lama pengalaman.
- `@Override public double hitungGaji()` => implementasi wajib kedua. Menghitung gaji berdasarkan gaji pokok (8 juta) ditambah tunjangan spesialis dan tunjangan pengalaman. Angka `8_000_000` menggunakan underscore sebagai pemisah ribuan agar mudah dibaca, namun tidak mempengaruhi nilai. Ekspresi `spesialisasi.equals("Umum") ? 0 : 3_000_000` adalah ternary operator — jika spesialisasi "Umum" maka tunjangan 0, selain itu 3 juta.
- `public void periksaPasien(String namaPasien)` dan `public void periksaPasien(String namaPasien, String keluhan)` => penerapan Method Overloading — dua method dengan nama sama namun parameter berbeda. Versi pertama hanya menerima nama pasien, versi kedua menerima nama pasien dan keluhan. Java menentukan versi mana yang dipanggil berdasarkan argumen yang diberikan. `getNama()` digunakan karena atribut `nama` di TenagaMedis bersifat `private` sehingga tidak bisa diakses langsung.
- `@Override public void perkenalan()` => penerapan Method Overriding — mendefinisikan ulang method `perkenalan()` dari TenagaMedis dengan format khusus Dokter yang mencantumkan gelar "Dr." dan spesialisasinya.
- `@Override public String toString()` => memanggil `super.toString()` untuk mendapatkan info dasar dari TenagaMedis, lalu menambahkan info spesifik Dokter dan gaji di bawahnya. `String.format("%,.0f", hitungGaji())` memformat angka gaji dengan pemisah ribuan (,) dan tanpa angka desimal (.0f).

5.7 Class Perawat.java

Class Perawat adalah subclass dari TenagaMedis yang merepresentasikan tenaga perawat. Memiliki struktur serupa dengan Dokter namun dengan atribut dan logika gaji yang berbeda sesuai karakteristik perawat.

Penjelasan Kode Program:

- `package model.tenaga` => berada di package yang sama dengan TenagaMedis, tidak perlu import.
- `public class Perawat extends TenagaMedis` => Perawat mewarisi TenagaMedis dan wajib mengimplementasikan `getInfoSpesifik()` dan `hitungGaji()`.
- `private String nomorSIP, private String unitKerja` => atribut khusus Perawat. nomorSIP adalah Surat Izin Praktik perawat. unitKerja menyimpan unit tempat perawat bertugas seperti ICU atau UGD.
- `private static int totalPerawat = 0` => counter khusus untuk jumlah objek Perawat yang dibuat.
- `public Perawat(String nama, ...) { super(..., "Perawat"); }` => Constructor memanggil `super()` dengan jabatan di-hardcode "Perawat", lalu mengisi atribut khusus Perawat. `totalPerawat++` dijalankan setiap objek baru dibuat.
- `@Override public String getInfoSpesifik()` => implementasi wajib yang mengembalikan nomor SIP dan unit kerja perawat.
- `@Override public double hitungGaji()` => implementasi wajib dengan perhitungan gaji Perawat yaitu gaji pokok 5 juta ditambah tunjangan 1 juta, total tetap 6 juta (belum mempertimbangkan variabel seperti Dokter).
- `public void rawatPasien(String namaPasien)` => method unik milik Perawat untuk mensimulasikan aksi merawat pasien. Menggunakan `getNama()` karena atribut nama di superclass bersifat private.
- `@Override public void perkenalan()` => override method perkenalan dengan format khusus Perawat yang menyebutkan unit kerjanya.
- `@Override public String toString()` => memanggil `super.toString()` lalu menambahkan info spesifik perawat dan gaji dengan format Rupiah.

5.8 Class Bidan.java

Class Bidan adalah subclass dari TenagaMedis yang merepresentasikan tenaga bidan. Keunikan kelas ini adalah memiliki atribut `jumlahPersalinan` yang nilainya bisa bertambah secara dinamis selama program berjalan, sehingga berpengaruh langsung pada perhitungan gaji.

Penjelasan Kode Program:

- `package model.tenaga` => berada di package yang sama dengan TenagaMedis.
- `public class Bidan extends TenagaMedis` => Bidan mewarisi TenagaMedis dan wajib mengimplementasikan kedua method abstraknya.
- `private String nomorSIB, private int jumlahPersalinan` => atribut khusus Bidan. nomorSIB adalah Surat Izin Bidan. jumlahPersalinan bertipe int untuk mencatat

berapa kali bidan menangani persalinan, dan diinisialisasi dengan nilai 0 di constructor.

- `private static int totalBidan = 0` => counter khusus jumlah objek Bidan.
- `public Bidan(String nama, ...) { super(..., "Bidan"); this.jumlahPersalinan = 0; }`
=> Constructor menginisialisasi jumlahPersalinan ke 0 karena bidan baru belum menangani persalinan apapun. `totalBidan++` dijalankan otomatis.
- `@Override public String getInfoSpesifik()` => mengembalikan nomor SIB dan jumlah persalinan yang sudah ditangani.
- `@Override public double hitungGaji()` => perhitungan gaji bersifat dinamis — gaji pokok 5,5 juta ditambah bonus per persalinan (`jumlahPersalinan * 100_000`). Artinya semakin banyak persalinan yang ditangani, semakin besar gajinya.
- `public void tanganiPersalinan(String namaPasien)` => method khusus Bidan. Setiap kali method ini dipanggil, `jumlahPersalinan++` dijalankan sehingga counter bertambah dan secara otomatis mempengaruhi hasil `hitungGaji()`.
- `@Override public void perkenalan()` => override perkenalan dengan format khusus Bidan yang menyebutkan nomor SIB-nya.
- `@Override public String toString()` => memanggil `super.toString()` dan menambahkan info spesifik serta gaji Bidan.

5.9 Class TenagaAdministrasi.java

Class `TenagaAdministrasi` adalah subclass dari `TenagaMedis` yang merepresentasikan tenaga non-medis yang bertugas di bagian administrasi rumah sakit.

Penjelasan Kode Program:

- `package model.tenaga` => berada di package yang sama dengan `TenagaMedis`.
- `public class TenagaAdministrasi extends TenagaMedis` => mewarisi `TenagaMedis` dan wajib mengimplementasikan kedua method abstraknya.
- `private String bagian, private String keahlian` => atribut khusus `TenagaAdministrasi`. `bagian` menyimpan divisi tempat bertugas seperti "Keuangan" atau "Rekam Medis". `keahlian` menyimpan kompetensi utama seperti "Akuntansi" atau "Sistem Informasi".
- `private static int totalAdmin = 0` => counter khusus jumlah objek `TenagaAdministrasi`.
- `public TenagaAdministrasi(String nama, ...) { super(..., "Tenaga Administrasi"); }`
=> Constructor dengan jabatan di-hardcode "Tenaga Administrasi", lalu mengisi atribut khusus. `totalAdmin++` dijalankan otomatis.
- `@Override public String getInfoSpesifik()` => mengembalikan informasi bagian dan keahlian tenaga administrasi.
- `@Override public double hitungGaji()` => gaji tetap flat 4 juta tanpa variabel tambahan, mencerminkan struktur gaji administrasi yang lebih sederhana.
- `public void inputData(String jenisData)` => method khusus `TenagaAdministrasi` untuk mensimulasikan aksi menginput data. Menggunakan `getNama()` karena nama di superclass bersifat private.

- `@Override public void perkenalan()` => override perkenalan dengan format yang menyebutkan bagian tugasnya.
- `@Override public String toString()` => memanggil `super.toString()` dan menambahkan info spesifik serta gaji administrasi.

5.10 Class Departemen.java

Class Departemen adalah kelas biasa (bukan abstract) yang merepresentasikan departemen atau divisi di dalam rumah sakit. Kelas ini memanfaatkan ArrayList untuk menyimpan daftar anggota tenaga medis secara dinamis.

Penjelasan Kode Program:

- `package model` => berada di package model (satu level di atas package tenaga dan ruangan).
- `import model.tenaga.TenagaMedis` => karena menggunakan class TenagaMedis yang berada di package berbeda, harus di-import terlebih dahulu.
- `import java.util.ArrayList` => import class ArrayList dari library bawaan Java untuk digunakan sebagai struktur data dinamis.
- `private String kodeDept, private String namaDept, private String kepalaDept` => atribut dasar departemen bersifat private.
- `private ArrayList<TenagaMedis> anggota` => atribut bertipe ArrayList yang menyimpan objek-objek TenagaMedis. `<TenagaMedis>` adalah generics yang membatasi ArrayList hanya boleh menyimpan objek TenagaMedis atau subclassnya (Dokter, Perawat, dst). Ini memanfaatkan konsep Polymorphism karena satu ArrayList bisa menyimpan berbagai jenis subclass TenagaMedis sekaligus.
- `public Departemen(String kodeDept, ...) { this.anggota = new ArrayList<>(); }` => Constructor menginisialisasi semua atribut dan membuat ArrayList kosong baru `new ArrayList<>()` siap menampung anggota.
- `public void tambahAnggota(TenagaMedis tenaga)` => method untuk menambahkan objek TenagaMedis ke dalam ArrayList menggunakan `anggota.add(tenaga)`. Karena parameternya bertipe TenagaMedis, method ini bisa menerima Dokter, Perawat, Bidan, maupun TenagaAdministrasi sekaligus — ini adalah penerapan Polymorphism.
- `public void tampilkanAnggota()` => method untuk mencetak daftar semua anggota departemen. `anggota.size()` mengembalikan jumlah elemen dalam ArrayList. Perulangan `for (TenagaMedis t : anggota)` adalah enhanced for loop untuk mengiterasi setiap anggota dan mencetak nama serta jabatannya menggunakan `t.getNama()` dan `t.getJabatan()`.
- `Getter` => `getAnggota()` mengembalikan seluruh ArrayList bertipe `ArrayList<TenagaMedis>` agar bisa diakses dari luar kelas.
- `@Override public String toString()` => menampilkan ringkasan departemen berupa kode, nama, kepala, dan jumlah anggota.

5.11 Class RumahSakit.java

Class RumahSakit adalah kelas utama yang merangkum seluruh entitas rumah sakit. Kelas ini menggunakan banyak ArrayList untuk mengelola berbagai jenis data dan menerapkan Method Overloading pada method penambahan tenaga medis.

Penjelasan Kode Program:

- `package model` => berada di package model.
- `import model.tenaga.*` dan `import model.ruangan.*` => tanda * berarti mengimport semua class yang ada di dalam package tersebut sekaligus, sehingga tidak perlu import satu per satu.
- `private final String namaRS` => atribut dengan modifier final artinya nilainya tidak bisa diubah setelah diinisialisasi pertama kali di constructor. Ini memastikan nama rumah sakit bersifat permanen dan tidak bisa diganti sembarangan.
- `private String alamat`, `private String noTelepon`, `private String tipeRS` => atribut informasi umum rumah sakit bersifat private.
- `private ArrayList<Departemen>`, `private ArrayList<Ruangan>`, `private ArrayList<Dokter>`, `private ArrayList<Perawat>`, `private ArrayList<Bidan>`, `private ArrayList<TenagaAdministrasi>` => enam ArrayList terpisah untuk menampung masing-masing jenis data. `ArrayList<Ruangan>` bisa menampung semua subclass Ruangan (Kantor, RuangRawatInap, RuangTindakan) berkat Polymorphism.
- `public RumahSakit(String namaRS, ...) { daftarDepartemen = new ArrayList<>(); ... }` => Constructor menginisialisasi semua atribut dan membuat enam ArrayList kosong sekaligus siap menampung data.
- `public void tambahTenagaMedis(Dokter d)`, `public void tambahTenagaMedis(Perawat p)`, `public void tambahTenagaMedis(Bidan b)`, `public void tambahTenagaMedis(TenagaAdministrasi ta)` => penerapan Method Overloading — empat method dengan nama yang sama namun parameter berbeda. Java secara otomatis memilih versi yang tepat berdasarkan tipe argumen yang diberikan, lalu menambahkan ke ArrayList yang sesuai.
- `public void tambahDepartemen(Departemen d)` dan `public void tambahRuangan(Ruangan r)` => method untuk menambahkan departemen dan ruangan ke ArrayList masing-masing. Parameter Ruangan r bisa menerima semua subclassnya berkat Polymorphism.
- `public void tampilkanProfil()` => method utama yang mencetak profil lengkap rumah sakit. Menggunakan `"=".repeat(65)` untuk membuat garis pemisah sepanjang 65 karakter. Kemudian iterasi setiap ArrayList menggunakan enhanced for loop untuk mencetak data masing-masing kategori. Method `toString()` dari setiap objek dipanggil secara otomatis saat objek di-print.
- `public String getNamaRS()` => satu-satunya getter untuk namaRS. Tidak ada setter karena namaRS bersifat final.

- `@Override public String toString()` => menampilkan ringkasan singkat rumah sakit berupa nama, alamat, dan tipe.

5.12 Class Main.java

Class Main adalah entry point (titik masuk) program. Tidak memiliki package karena berada di luar folder model, dan berisi seluruh logika untuk membuat objek, menghubungkan data, dan mendemonstrasikan konsep PBO.

Penjelasan Kode Program:

- `import model.*; import model.tenaga.*; import model.ruangan.*` => karena Main berada di luar semua package, semua class dari package model, tenaga, dan ruangan harus di-import agar bisa digunakan.
- `public class Main` => tidak memiliki deklarasi package karena berada di root folder program, di luar folder model.
- `public static void main(String[] args)` => method utama yang dijalankan otomatis pertama kali oleh JVM saat program dieksekusi. static berarti bisa dijalankan tanpa membuat objek Main. `String[] args` adalah array untuk menampung argumen dari command line (tidak digunakan dalam program ini).
- `RumahSakit rs1 = new RumahSakit(...)` => membuat objek RumahSakit pertama dengan memanggil constructor yang menerima 4 parameter: nama, alamat, nomor telepon, dan tipe RS.
- `Dokter dr1 = new Dokter("Budi Santoso", "NIP001", 45, "Laki-laki", "Kardiologi", "STR-001", 15)` => membuat objek Dokter dengan 7 argumen. Di balik layar, constructor Dokter memanggil `super()` yang memanggil constructor `TenagaMedis`, sehingga `totalTenagaMedis++` dan `totalDokter++` keduanya bertambah.
- `bd1.tanganiPersalinan("Ny. Sari")` => memanggil method khusus Bidan sebelum data ditambahkan ke RS, sehingga `jumlahPersalinan` bertambah dan mempengaruhi gaji bidan saat dihitung nanti.
- `rs1.tambahTenagaMedis(dr1), rs1.tambahTenagaMedis(pr1), dst` => memanggil method overloading `tambahTenagaMedis()`. Java otomatis memilih versi Dokter, Perawat, atau lainnya sesuai tipe objek yang diberikan.
- `TenagaMedis[] semuaTenaga = {dr1, pr1, bd1, ta1}` => membuat array bertipe `TenagaMedis` yang diisi dengan berbagai subclassnya sekaligus. Ini adalah penerapan Polymorphism — satu array bisa menampung Dokter, Perawat, Bidan, dan `TenagaAdministrasi` karena semuanya adalah `TenagaMedis`.
- `for (TenagaMedis t : semuaTenaga) t.perkenalan()` => perulangan enhanced for yang memanggil `perkenalan()` pada setiap elemen. Meskipun tipe variabel `t` adalah `TenagaMedis`, Java menjalankan versi `perkenalan()` sesuai kelas asli objek tersebut inilah inti dari Runtime Polymorphism.
- `Ruangan[] semuaRuangan = {kt1, rri1, rt1}` => sama seperti array `TenagaMedis`, array bertipe `Ruangan` bisa menampung semua subclassnya. Saat

r.gunakanRuangan() dipanggil, versi yang dijalankan sesuai kelas asli objek (Kantor, RuangRawatInap, atau RuangTindakan).

- `TenagaMedis.getTotalTenagaMedis()`, `Dokter.getTotalDokter()`, `dst` => pemanggilan method static langsung dari nama kelas tanpa membuat objek, membuktikan bahwa counter berjalan dengan benar selama program.
- `rs1.getNamaRS()` dan `rs2.getNamaRS()` => membuktikan konsep final nilai nama RS hanya bisa dibaca, tidak bisa diubah setelah objek dibuat.

5.13 Konsep PBO

- Abstraction (Abstraksi)

Abstraction diterapkan melalui dua abstract class yaitu `TenagaMedis` dan `Ruangan`. Kedua kelas ini tidak bisa dibuat objeknya secara langsung karena bersifat abstract, dan hanya berfungsi sebagai kerangka umum yang mendefinisikan struktur dasar untuk subclass-subclassnya. Pada class `TenagaMedis`, terdapat dua method abstract yaitu `getInfoSpesifik()` dan `hitungGaji()`. Kedua method ini sengaja dibuat tanpa isi karena setiap jenis tenaga medis memiliki informasi spesifik dan perhitungan gaji yang berbeda-beda. Subclass wajib mengimplementasikan sendiri sesuai karakteristiknya masing-masing. Pada class `Ruangan`, terdapat dua method abstract yaitu `getJenisRuangan()` dan `gunakanRuangan()`. Method `gunakanRuangan()` dibuat abstract karena cara penggunaan setiap ruangan berbeda — Kantor langsung mengubah status, `RuangRawatInap` memantau kapasitas tempat tidur, dan `RuangTindakan` menggunakan logika boolean untuk memastikan tidak ada dua tindakan sekaligus.

- Inheritance (Pewarisan)

Inheritance diterapkan dalam dua hierarki pewarisan yang berbeda dalam program ini.

Hierarki pertama adalah `TenagaMedis` sebagai superclass yang diwarisi oleh empat subclass yaitu `Dokter`, `Perawat`, `Bidan`, dan `TenagaAdministrasi`. Setiap subclass secara otomatis mewarisi atribut `nama`, `nip`, `umur`, `jenisKelamin`, dan `jabatan` dari `TenagaMedis`, sehingga tidak perlu mendefinisikan ulang atribut-atribut tersebut. Masing-masing subclass kemudian menambahkan atribut dan method yang spesifik sesuai profesinya. Hierarki kedua adalah `Ruangan` sebagai superclass yang diwarisi oleh tiga subclass yaitu `Kantor`, `RuangRawatInap`, dan `RuangTindakan`. Setiap subclass mewarisi atribut dasar ruangan seperti `kodeRuangan`, `namaRuangan`, `antai`, `kapasitas`, dan `statusRuangan`, lalu menambahkan atribut khusus sesuai jenis ruangnya. Pewarisan dilakukan menggunakan keyword `extends`, dan setiap constructor subclass memanggil `super()` di baris pertama untuk menginisialisasi atribut yang diwarisi dari superclass.

- Encapsulation (Enkapsulasi)

Encapsulation diterapkan secara konsisten di seluruh kelas dalam program ini melalui penggunaan access modifier dan method getter-setter. Sebagian besar atribut di setiap kelas dideklarasikan sebagai private sehingga tidak bisa diakses langsung dari luar kelas. Akses terhadap atribut tersebut hanya bisa dilakukan melalui method getter untuk membaca nilai dan setter untuk mengubah nilai. Contohnya pada class TenagaMedis, atribut nama, nip, umur, dan jenisKelamin bersifat private dan hanya bisa diakses melalui getName(), setName(), dan seterusnya. Selain private, program ini juga menggunakan access modifier protected pada atribut jabatan di TenagaMedis dan statusRuangan di Ruangan. Atribut protected dipilih agar subclass bisa mengakses dan mengubah nilai tersebut secara langsung tanpa perlu melalui setter, misalnya statusRuangan = "Digunakan" pada class Kantor dan RuangTindakan. Program ini juga menerapkan final pada atribut namaRS di class RumahSakit. Dengan final, nilai nama rumah sakit tidak bisa diubah setelah diinisialisasi pertama kali di constructor, sehingga data tersebut terlindungi dari perubahan yang tidak diinginkan.

- Polymorphism (Polimorfisme)

Polymorphism diterapkan dalam dua bentuk yaitu method overriding dan method overloading.

Method Overriding diterapkan pada method perkenalan(), hitungGaji(), getInfoSpesifik(), dan toString() di seluruh subclass TenagaMedis, serta method gunakanRuangan(), getJenisRuangan(), dan toString() di seluruh subclass Ruangan. Setiap subclass mendefinisikan ulang method-method tersebut dengan implementasi yang berbeda sesuai karakteristiknya. Efek nyata dari overriding ini terlihat pada class Main ketika array TenagaMedis[] semuaTenaga diisi dengan berbagai subclass lalu perkenalan() dipanggil — setiap objek menjalankan versi perkenalan() miliknya sendiri meskipun dipanggil melalui referensi TenagaMedis. Hal yang sama terjadi pada array Ruangan[] semuaRuangan ketika gunakanRuangan() dipanggil, masing-masing ruangan bereaksi berbeda sesuai logika yang sudah didefinisikan.

Method Overloading diterapkan pada dua tempat. Pertama pada class Dokter dengan method periksaPasien() yang hadir dalam dua versi — versi satu parameter hanya menerima nama pasien, dan versi dua parameter menerima nama pasien beserta keluhannya. Kedua pada class RumahSakit dengan method tambahTenagaMedis() yang hadir dalam empat versi sesuai tipe parameter yang diterima yaitu Dokter, Perawat, Bidan, dan TenagaAdministrasi. Java secara otomatis memilih versi yang tepat berdasarkan tipe argumen yang diberikan saat pemanggilan.

- Static

Konsep static diterapkan pada atribut counter di setiap kelas tenaga medis dan ruangan. Atribut `totalTenagaMedis` di `TenagaMedis`, `totalDokter` di `Dokter`, `totalPerawat` di `Perawat`, `totalBidan` di `Bidan`, `totalAdmin` di `TenagaAdministrasi`, dan `totalRuangan` di `Ruangan` semuanya bersifat static. Karena static, atribut-atribut ini dimiliki oleh kelas bukan oleh objek, sehingga nilainya dibagi bersama oleh semua objek dari kelas tersebut. Setiap kali objek baru dibuat melalui constructor, nilai counter bertambah otomatis. Nilai counter ini kemudian bisa diakses langsung melalui nama kelas tanpa perlu membuat objek, misalnya `TenagaMedis.getTotalTenagaMedis()` dan `Ruangan.getTotalRuangan()`.

- Constructor dan Keyword `this` & `super`

Constructor diterapkan di setiap kelas untuk menginisialisasi nilai atribut secara otomatis saat objek dibuat. Keyword `this` digunakan di dalam constructor untuk membedakan antara atribut kelas dengan parameter yang memiliki nama sama, misalnya `this.nama = nama`. Keyword `super` digunakan di setiap constructor subclass untuk memanggil constructor superclass di baris pertama. Misalnya constructor `Dokter` memanggil `super(nama, nip, umur, jenisKelamin, "Dokter")` untuk menginisialisasi atribut yang diwarisi dari `TenagaMedis`. Hal ini memastikan bahwa atribut superclass selalu terinisialisasi dengan benar sebelum atribut subclass diisi.

- ArrayList dan Generics

`ArrayList` digunakan secara luas dalam program ini untuk menyimpan banyak objek secara dinamis. Class `Departemen` menggunakan `ArrayList<TenagaMedis>` untuk menyimpan daftar anggota, sedangkan class `RumahSakit` menggunakan enam `ArrayList` sekaligus untuk menyimpan daftar dokter, perawat, bidan, tenaga administrasi, departemen, dan ruangan. Penggunaan generics `<TenagaMedis>` dan `<Ruangan>` pada `ArrayList` memberikan keuntungan ganda. Pertama, tipe data yang disimpan terjamin keamanannya karena hanya objek bertipe sesuai yang bisa masuk. Kedua, `ArrayList` bertipe superclass seperti `ArrayList<TenagaMedis>` dan `ArrayList<Ruangan>` dapat menampung semua subclassnya sekaligus berkat Polymorphism, sehingga satu `ArrayList` saja sudah cukup untuk menampung `Dokter`, `Perawat`, `Bidan`, dan `TenagaAdministrasi` secara bersamaan.

6. Penjelasan Cara Running Program

Untuk menjalankan program, langkah pertama yang harus dilakukan adalah navigasi ke folder tempat kode disimpan melalui CMD menggunakan perintah:

```
cd C:\Users\Bestdym\Data\Praktikum\TugasPBO
```

Perintah ini wajib dilakukan agar CMD mengetahui lokasi file program yang akan dikompilasi dan dijalankan. Setelah berada di dalam folder TugasPBO, langkah selanjutnya adalah kompilasi, yaitu menerjemahkan kode Java menjadi file .class yang dapat dimengerti oleh komputer. Terdapat dua cara kompilasi yang dapat dilakukan:

Cara pertama adalah kompilasi sekaligus, yaitu mengompilasi seluruh file .java dalam satu perintah menggunakan:

```
javac Model/Ruangan/*.java Model/Tenaga/*.java Model/*.java Main.java
```

Perintah ini mengompilasi semua file secara berurutan mulai dari package Ruangan, kemudian Tenaga, lalu Model, dan terakhir Main.java. Urutan ini penting karena Java membutuhkan superclass ter-compile lebih dulu sebelum subclass-nya bisa dikompilasi. Cara ini lebih cepat dan efisien karena cukup satu baris perintah untuk mengompilasi seluruh file sekaligus.

Cara kedua adalah kompilasi satu per satu, yaitu mengompilasi setiap file secara berurutan dari kelas yang paling atas hierarkinya hingga ke bawah:

```
javac Model/Ruangan/Ruangan.java
```

```
javac Model/Ruangan/Kantor.java
```

```
javac Model/Ruangan/RuangRawatInap.java
```

```
javac Model/Ruangan/RuangTindakan.java
```

```
javac Model/Tenaga/TenagaMedis.java
```

```
javac Model/Tenaga/Dokter.java
```

```
javac Model/Tenaga/Perawat.java
```

```
javac Model/Tenaga/Bidan.java
```

```
javac Model/Tenaga/TenagaAdministrasi.java
```

```
javac Model/Departemen.java
```

```
javac Model/RumahSakit.java
```

```
javac Main.java
```

Urutan ini wajib diikuti karena setiap kelas bergantung pada kelas di atasnya — misalnya Dokter membutuhkan TenagaMedis sudah ter-compile lebih dulu, dan RumahSakit membutuhkan semua kelas lainnya sudah siap. Cara ini lebih

direkomendasikan saat belajar karena jika terjadi error, pesan error hanya muncul dari file yang bermasalah sehingga lebih mudah dilacak. Setelah proses kompilasi selesai tanpa error, program dieksekusi menggunakan perintah:

`java Main`

Perintah ini menjalankan class Main yang di dalamnya terdapat method main() sebagai titik masuk eksekusi program. Program kemudian akan menampilkan output berupa profil lengkap dua rumah sakit beserta daftar tenaga medis, departemen, dan ruangnya, dilanjutkan dengan demonstrasi berbagai konsep PBO yang diterapkan dalam program seperti polymorphism, method overloading, penggunaan static, dan konsep final.

7. Lampiran

Akses GitHub : https://github.com/bestdym/ASN2_PBO